

Escalonamento de tarefas com a equação Navier-Stokes

Para essa atividade, compreendi que é necessário realizar a simulação do movimento de um fluido ao longo do tempo considerando apenas os efeitos de viscosidade da equação de Navier-Stokes. Desconsiderando pressão e forças externas, o modelo se reduz a uma equação de difusão pura.

A atividade exige o uso de diferenças finitas para discretizar o espaço (2D), a validação da estabilidade com um campo constante, a criação de uma perturbação para observar sua difusão suave e, por fim, a paralelização com OpenMP explorando as cláusulas `schedule` e `collapse`.

I. Estrutura do Grid e Alocação Dedicada

Para garantir uma implementação fortemente tipada e evitar o overhead de alocações dinâmicas (`malloc/free`) dentro do laço temporal de alta performance, optou-se pela estratégia de **Double Buffering** (Ping-Pong). Uma estrutura dedicada gerencia a matriz bidimensional (lista de listas):

```
#include <omp.h>
#include <stdio.h>
#include <stdlib.h>

#define NX 64
#define NY 64
#define NT 5000
#define DX 0.1f
#define DY 0.1f
#define DT 0.001f
#define VISC 0.1f

typedef struct {
    int nx;
    int ny;
    float **data;
} Grid2D;

Grid2D alocar_grid(int nx, int ny, float valor_base) {
    Grid2D grid = {
        .nx = nx,
        .ny = ny,
        .data = (float **)malloc(nx * sizeof(float *)),
    };

    for (int i = 0; i < nx; i++) {
        grid.data[i] = (float *)malloc(ny * sizeof(float));
    }

    for (int i = 0; i < grid.nx; i++) {
        for (int j = 0; j < grid.ny; j++) {
            grid.data[i][j] = valor_base;
        }
    }
}
```

```

}

int cx = grid.nx / 2, cy = grid.ny / 2, r = 4;
for (int i = cx - r; i <= cx + r; i++) {
    for (int j = cy - r; j <= cy + r; j++) {
        grid.data[i][j] = 10.0f;
    }
}

return grid;
}

```

II. Função Pura de Diferenças Finitas

Para garantir a separação explícita de entradas e saídas, a etapa de diferenças finitas foi isolada em uma função pura. Ela recebe o estado atual em modo leitura (const) e escreve as atualizações do Laplaciano discreto de segunda ordem no buffer de saída, sem gerar efeitos colaterais de concorrência ou dependências de dados.

```

void calcular_proximo_passo(const Grid2D u_current, const Grid2D u_next,
                           const float nu, const float dt, const float dx,
                           const float dy) {
    const int nx = u_current.nx;
    const int ny = u_current.ny;
    const float idx2 = 1.0f / (dx * dx);
    const float idy2 = 1.0f / (dy * dy);

#pragma omp parallel for collapse(2) schedule(static, 16)
    for (int i = 1; i < nx - 1; i++) {
        for (int j = 1; j < ny - 1; j++) {
            const float laplaciano =
                (u_current.data[i + 1][j] - 2.0f * u_current.data[i][j] +
                 u_current.data[i - 1][j]) *
                idx2 +
                (u_current.data[i][j + 1] - 2.0f * u_current.data[i][j] +
                 u_current.data[i][j - 1]) *
                idy2;

            u_next.data[i][j] = u_current.data[i][j] + dt * nu * laplaciano;
        }
    }
}

```

III. Execução Principal e Simulação da Perturbação

O programa principal inicializa o fluido em calmaria (velocidade constante igual a 1.0) e gera uma perturbação energética no centro do domínio. O avanço temporal é computado alternando as referências de leitura e escrita a cada iteração de forma instantânea (operadores de rotação de ponteiros), eliminando realocações de memória.

```

int main(void) {
    Grid2D u_velA = alocar_grid(NX, NY);
    Grid2D u_velB = alocar_grid(NX, NY);
}

```

```

Grid2D u_atual = u_velA;
Grid2D u_proximo = u_velB;

printf("Velocidade inicial no centro: %.2f\n", u_atual.data[NX / 2][NY / 2]);

double tempo_inicio = omp_get_wtime();

for (int t = 0; t < NT; t++) {
    calcular_proximo_passo(u_atual, u_proximo, VISC, DT, DX, DY);

    // Estratégia Ping-Pong: Inversão dos papéis dos buffers alocados
    Grid2D temp = u_atual;
    u_atual = u_proximo;
    u_proximo = temp;
}

double tempo_fim = omp_get_wtime();

printf("Velocidade final no centro apos %d passos: %f\n", NT, u_atual.data[NX / 2]
[NY / 2]);
printf("Tempo total gasto: %f segundos\n", tempo_fim - tempo_inicio);

liberar_grid(u_velA);
liberar_grid(u_velB);

return EXIT_SUCCESS;
}

```